HANDS-ON 4 · DAY 2 / MORNING · GROUP'S SHEET

Lion or tiger: what can 58 photos teach a pretrained network?

The case. A firm wants its photos sorted into classes (products, defects, species) but holds only a few dozen labelled examples of each. Three questions: can it use a network already trained on ImageNet, training only a new head? Can it follow how that training goes? And how far can the test score be trusted?

DATA CARD · LION-TIGER-TRAIN.ZIP, LION-TIGER-TEST.ZIP

What a row is. One photo, a JPEG file in a folder; the network reads its pixels and nothing else. 58 training photos and 56 test photos. The label is the word before the underscore of the file name: `lion_001.jpg`, `tiger_001.jpg`.

The target. `label`, `lion` or `tiger`. Training: 30 lions and 28 tigers. Test: 38 lions and 18 tigers, so always saying `lion` is right 68% of the time, and one test photo is worth 1.79 points.

How the photos were chosen. Taken from the web for a course exercise. No source, photographer or licence is recorded with them. They are not a sample of any population of photos.

Its history. The inherited test set held 65 photos, 8 of them byte-identical copies of training photos: they were removed, and one more was withdrawn. Two training photos carried the other animal's label: they were corrected. Every photo was renamed `<label>_<nnn>.jpg`, so that one rule reads all the labels.

Source and licence. Lions and tigers: web photos, licences unknown, used in class and never republished. Cats, in panel 5: the cat class of Natural Images, P. Roy, S. Ghosh, S. Bhattacharya and U. Pal (2018), Effects of degradations on deep neural network architectures, arXiv:1807.10108; 120 training and 80 test photos of its 885. The licence of its public copy is not verified.

archive	photos	lions	tigers	cats	dataset after step 1, used in
`lion-tiger-train.zip`	58	30	28		`lion-tiger-train`, step 2
`lion-tiger-test.zip`	56	38	18		`lion-tiger-test`, step 3
`cats-train.zip`	120			120	`cats-train`, panel 5
`cats-test.zip`	80			80	`cats-test`, panel 5

STEP 1 · LOAD THE PHOTOS AND THEIR LABELS

1 ☐ **Upload the four archives into one folder**
In a new project, on the Flow: **+ Dataset** → **Folder**, the one under **Dataiku Managed** (not **Files in Folder**), named `dataset`. Open it, upload the four zips as they are, and decompress each where Dataiku offers it: one subfolder per archive.

2 ☐ **List the files**
Back on the Flow, click the folder `dataset` once; in the right panel, **List Contents**, output `paths`; **Run**. One row per photo of every subfolder, with its `path` and its `basename` (the file name).

Rows: _____

3 ☐ **Read the label and the archive**
On `paths`: **Prepare**, output `labels`, a step **Extract with regular expression** on `basename`, pattern `^([a-z]+)_[0-9]+`, output column `label`; **Run**. On `labels`: **Prepare**, output `sets`, the same step on `path`, pattern `([a-z-]+)/[^/]+$` (the subfolder), output column `set`; **Run**.

4 ☐ **One dataset per archive**
On `sets`: **Split**, **Map values of a single column** on `set`, four outputs named as its values: `lion-tiger-train`, `lion-tiger-test`, `cats-train`, `cats-test`; **Run**.

In `lion-tiger-train`, lions: _____ tigers: _____

STEP 2 · TRAIN A NEW HEAD ON A PRETRAINED NETWORK

1 ☐ **Create the analysis**
Select `lion-tiger-train`, then **Lab** → **Image classification**: target `label`, the images at `path` in the folder `dataset`.

2 ☐ **Reuse it, do not fine-tune**
Design → **Train/Test set**: **Split the dataset**, train ratio 0.8. **Design** → **Training**: **Pretrained model Lighter / Faster (EfficientNet B0)**, **Number of fine-tuned layers** 0; the rest at its default (50 epochs, early stopping). **Train**.

Training time: _____ epochs run: _____

3 ☐ **Follow the training, then read its own test**
Results: the training curve, if shown, then the confusion matrix; click a cell to see its photos in the image feed.

Accuracy: _____ on how many photos? _____

STEP 3 · SCORE PHOTOS IT HAS NEVER SEEN

1 ☐ **Deploy the model**
In the analysis, on the model: **Deploy**, model name `model_lions_tigers`. It appears on the Flow.

2 ☐ **Score them**
Click `lion-tiger-test` once; in the right panel, **Score**, with the model `model_lions_tigers` and the folder `dataset`, output `lion-tiger-test_scored`; **Run**.

3 ☐ **Count the errors**
On `lion-tiger-test_scored`: **Prepare**, output `lion-tiger-test_scored_errors`, a step **Formula** into `check`: `if(label == prediction, "right", "wrong")`; **Run**.

Wrong: _____ of 56 accuracy: _____ lion-always baseline: 68%

What do the wrong photos have in common? Is any of them wrongly labelled?

4 · FINE-TUNE TWO LAYERS, AND COMPARE

1 ☐ **Unlock two layers**
In the same analysis: **Design** → **Training**, **Number of fine-tuned layers** 2, the rest unchanged. **Train**.

2 ☐ **Score with the new model**
On the new model: **Deploy**, updating the existing model `model_lions_tigers`. On the Flow, open `model_lions_tigers`: the new version must be the active one, or the Score recipe still uses the old model. Run the Score recipe again and count as in step 3.3.

fine-tuned layers	training time	accuracy in Results	wrong of 56
0 (step 2)			
2			

Both accuracies come from Dataiku's own test split: how many photos is that, and what is one photo worth?

5 · ADD A THIRD, LARGER CLASS: CATS

1 ☐ **Stack the cats in**
Select `lion-tiger-train` and `cats-train`; in the right panel, **Stack**, output `felines-train`; **Run**. The same for `lion-tiger-test` and `cats-test`, output `felines-test`.

2 ☐ **Retrain**
An analysis is tied to its dataset, so a new one: on `felines-train`, as in steps 2.1 and 2.2, 0 fine-tuned layers; **Train**. Three classes now.

class	training photos	test photos	recall
lion	30	38	
tiger	28	18	
cat	120	80	

3 ☐ **Read the averages**
Results → **Metrics**: the averaged scores.

Macro average: _____ weighted average: _____

6 · DECIDE

Which model would the firm deploy, with 0 or 2 fine-tuned layers, and why? For the three classes, which score would you report, the macro or the weighted average?